Physics-Informed Neural Networks

From differential equations to the lid-driven cavity

FlowMLLab | Week 4.2 | Ehsan Roohi | Graduate lecture notes

This lecture connects the neural networks of Week 2 with the cavity solver and surrogates of Weeks 1 and 4. Its central question is not whether a network can draw a vortex. It is whether an approximate velocity and pressure field solves a precisely specified boundary-value problem, and what evidence would justify trusting that field. By the end, the reader should be able to construct a momentum residual, distinguish soft from hard constraints, derive a streamfunction lifting, and design an independent cavity validation experiment.

The first part develops PINNs without assuming previous experience with them. The second reads the square-cavity implementation in Christopher J. McDevitt's DeepPlasma repository [1] as a concrete research example. McDevitt has given Ehsan Roohi permission to use the cavity case for this course. The discussion and figures here are newly prepared; the external implementation remains linked to its original repository. No high-Reynolds-number PINN training result is claimed in these notes.

1. A familiar flow, a different numerical representation

Imagine a square container filled with an incompressible Newtonian fluid. Its top wall translates to the right, while the other walls remain fixed. Viscous stresses transfer momentum from the lid into the fluid. Because fluid cannot pass through the walls, the motion recirculates: the upper flow must turn, return through the interior, and close the circulation. Pressure participates in this redirection of momentum; it is not an externally prescribed field throughout the cavity.

The geometry is simple, but the numerical problem is not trivial. Wall gradients, recirculation, corner behavior, and nonlinear transport all matter. A visually plausible clockwise vortex can coexist with incorrect velocity magnitudes, pressure gradients, or wall stresses. This makes the cavity useful for learning the difference between an attractive visualization and a qualified numerical solution.

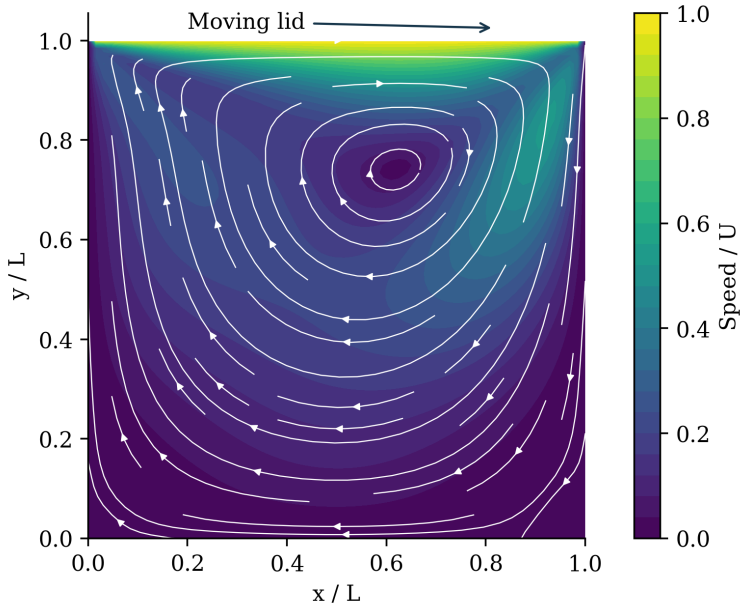


Figure 1. Retained conventional CFD field at Re=100, with speed contours and streamlines. This is a classical-lid FlowMLLab case, not a prediction from the regularized-lid PINN. Equal axis scales preserve the square geometry.

A conventional CFD solver represents the unknowns on a mesh and solves a discretized system. A supervised surrogate learns a map from previously computed input-output examples. A PINN instead represents the

unknown field by a differentiable neural network and adjusts its parameters to reduce differential-equation and constraint residuals. The physics supplies training information at locations where no measured velocity is available. Nevertheless, a finite set of residual evaluations is still a numerical approximation; the method has not escaped resolution, sampling, conditioning, or validation.

Table 1. Three different uses of a numerical representation.

Method	What determines the field?	Independent evidence
CFD solver	Discrete equations, boundary conditions and convergence.	Mesh/time refinement and matched benchmarks.
Supervised surrogate	Loss against previously generated fields.	Complete held-out cases and a non-neural baseline.
Forward PINN	PDE residual plus imposed constraints at fixed parameters.	Matched reference, boundary errors and unseen residual points.

The PINN construction was popularized for forward and inverse PDE problems by Raissi, Perdikaris, and Karniadakis [2]. A forward problem specifies the model parameters and boundary conditions, then seeks the field. An inverse problem uses observations to infer an unknown parameter or forcing together with the field. These are different information problems, even when implemented with similar networks. Here we first solve a forward, steady, two-dimensional problem with a fixed Reynolds number.

2. Specify the physical problem before the network

Let the dimensional cavity width be L, the characteristic lid speed U, the density rho, and the kinematic viscosity nu. Scale position by L, velocity by U, and pressure difference by rho U squared. Pressure is measured relative to an arbitrary reference because only its gradient enters the incompressible momentum equations. The dimensionless Reynolds number compares the chosen inertial and viscous scales.

$$x^{*}=x/L,\quad y^{*}=y/L,\quad (u^{*},v^{*})=(u,v)/U$$
$$p^{*}=(p-p_0)/(\rho U^2),\qquad Re=UL/\nu \tag{1}$$

Dropping the stars after nondimensionalization gives the steady incompressible equations on the unit square. The pressure terms have positive signs when all terms are moved to the left; the viscous terms have negative signs. Writing the residual in this convention is useful because the code can be checked term by term.

$$r_c=u_x+v_y=0$$
$$r_u=uu_x+vu_y+p_x-Re^{-1}(u_{xx}+u_{yy})=0 \tag{2}$$
$$r_v=uv_x+vv_y+p_y-Re^{-1}(v_{xx}+v_{yy})=0$$

Here u and v are the horizontal and vertical velocities. These equations assume constant density and viscosity, no body force, and a steady two-dimensional state. They are not RANS equations and contain no turbulence closure. A steady high-Re solution, if obtained, is not by itself evidence that a physical three-dimensional experiment remains steady or stable.

For the classical idealized cavity, the lid has u=1 and v=0 away from its endpoints; the stationary walls have u=v=0. At the two top corners these prescriptions meet discontinuously. One must document which condition is assigned to a corner grid point, whether the corner is excluded, or whether the lid is regularized. A network

with smooth activations cannot represent two different velocities at the same coordinate.

Re is a scale ratio, not a statement that the local convective term is exactly Re times the local viscous term. Near a thin wall layer, large second derivatives can make viscous effects important even when 1/Re is small. At a stagnation point the convective contribution may be small. Students should inspect the actual terms rather than infer their pointwise balance from Re alone.

3. A PINN is a field and a differentiable residual

Start with a coordinate network whose inputs are x and y and whose outputs are u, v, and p. A multilayer perceptron repeatedly applies affine maps and smooth nonlinear functions. Its parameters are shared across all spatial points: changing a weight changes the represented field throughout the domain. In a single-case PINN, coordinates identify a location, not a training example with a known velocity label.

$$\mathbf{h}^{(0)} = (x, y), \quad \mathbf{h}^{(k+1)} = \tanh(W_k \mathbf{h}^{(k)} + \mathbf{b}_k)$$
$$(u_\theta, v_\theta, p_\theta) = W_o \mathbf{h}^{(K)} + \mathbf{b}_o \tag{3}$$

Automatic differentiation applies the chain rule to the operations used to evaluate the network. It gives derivatives of the represented function, up to floating-point effects; it does not give the derivative of the unknown exact solution. A perfectly differentiated wrong field remains a wrong field. For a strong-form viscous residual, smooth activations such as tanh are natural because second spatial derivatives must be meaningful. A piecewise-linear ReLU velocity network has zero second derivatives almost everywhere and is not a straightforward substitute in this formulation.

There are two differentiation tasks. Spatial differentiation forms u_x, u_y, and the other derivatives entering the PDE. Parameter differentiation then forms the gradient of the loss with respect to the network weights. The graph for the spatial derivatives must remain available for that second task. Detaching a derivative before forming the loss can silently destroy the optimization problem even if its numerical value looks reasonable.

In a pointwise MLP, differentiating the sum of batch outputs gives the desired per-point derivatives because one sample's output does not depend on another sample's coordinates. This shortcut needs reconsideration for architectures that mix samples. Index conventions also matter: an array stored as field[y_index, x_index] is not interchangeable with field[x_index, y_index]. The derivative and orientation tests of Week 1.1 belong here before any expensive training.

4. The loss defines which problem is actually solved

Evaluate the PDE residuals at interior collocation points, and boundary mismatches at separately sampled wall points. For a direct velocity-pressure network, a basic objective combines the two momentum residuals, continuity, prescribed velocity, and a pressure gauge. The terms below are means over their own point sets; the weights therefore express priorities and scaling rather than accidentally reflecting point counts.

$$\mathrm{L} = \lambda_m \langle r_u^2 + r_v^2 \rangle_\Omega + \lambda_c \langle r_c^2 \rangle_\Omega$$
$$+ \lambda_b \langle \|\mathbf{u}_\theta - \mathbf{u}_b\|^2 \rangle_{\partial\Omega} + \lambda_g |p_\theta(\mathbf{x}_0)|^2 \tag{4}$$

The gauge term may impose p at one reference point or impose a zero spatial mean. Do not impose an arbitrary pressure distribution along all walls simply because pressure is an output. That would generally change or overconstrain the boundary-value problem. Pressure offsets should also be aligned before computing field errors against a reference.

Loss weights are not universal constants. Large boundary weights can enforce wall motion while leaving a poor interior solution; small weights can permit a nearly motionless field with deceptively small momentum residuals. Report each term separately, inspect the wall error directly, and select weighting rules on a development protocol rather than on the final reference comparison. Nondimensionalization comes before weight tuning.

No interior observation term is necessary to define this forward problem. If sparse CFD or measured values are added, call the experiment data-assisted and record their coordinates, uncertainty, and role. Keep evaluation observations out of fitting. A lower loss after adding reference data is not evidence of a data-free solver. Equally, data assistance is not a defect when it is part of the stated scientific task.

5. Enforce incompressibility by construction

For a smooth two-dimensional streamfunction, define horizontal velocity as its y derivative and vertical velocity as minus its x derivative. The divergence cancels because the mixed partial derivatives agree. The vorticity is minus the Laplacian of the streamfunction with this sign convention.

$$u = \psi_y, \quad v = -\psi_x, \quad u_x + v_y = \psi_{yx} - \psi_{xy} = 0$$
$$\omega_z = v_x - u_y = -\nabla^2 \psi \tag{5}$$

A streamfunction-pressure network thus needs only two raw outputs. Continuity is no longer a separate penalty competing with momentum. This is a structural constraint, not a reward for achieving a sufficiently small continuity loss. It remains worthwhile to check divergence independently on exported fields: differentiation, interpolation, transposition, or export can break an identity that held inside the network.

The cost is higher derivative order. Velocity already uses one derivative of the streamfunction; its viscous Laplacian uses three. The optimization gradient differentiates through this construction again with respect to parameters. A small parameter count therefore does not imply a cheap residual evaluation. Smoothness, precision, and memory use become particularly important.

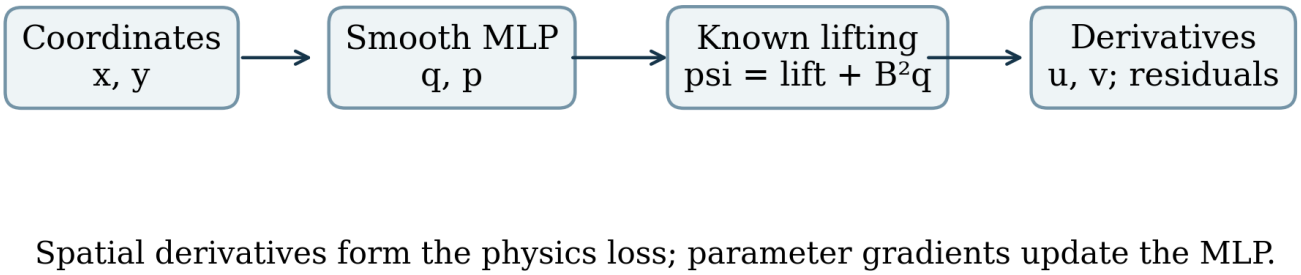


Figure 2. The streamfunction-pressure construction. The network supplies a correction, not the prescribed wall motion. Momentum residuals require third spatial derivatives of the physical streamfunction.

Hard constraints also require an appropriate function class. A streamfunction that is constant on a connected wall enforces no penetration there, but no slip additionally constrains its normal derivative. Multiplying a raw network by a factor that vanishes once on the wall does not automatically make that derivative vanish. The order of vanishing is the essential step in the construction below.

6. Deriving the DeepPlasma boundary lifting

The inspected implementation [1] uses a known lifting plus a trainable correction. Denote the first raw network output by q_theta and the second by p_theta. Define a boundary factor B and write the physical streamfunction as follows.

$$B = 16x(1-x)y(1-y), \qquad \psi_\theta = \psi_{\text{lid}} + B^2 q_\theta$$
$$u = \psi_{\text{lid},y} + 2BB_y q_\theta + B^2 q_{\theta,y} \tag{6}$$
$$v = -\psi_{\text{lid},x} - 2BB_x q_\theta - B^2 q_{\theta,x}$$

On every wall B=0. Since the derivative of B squared is 2B times the derivative of B, both the correction and its first derivatives vanish on a wall for a smooth finite network. Consequently, the trainable correction cannot change the prescribed wall velocities. All wall motion is supplied by the known lifting. This is why squaring the boundary factor matters.

The specific lifting has a smooth lid profile g(x), a polynomial y factor, and a Gaussian localization near the top wall. Its parameters below are dimensionless lengths, not CFD grid spacings, despite the variable names dx and dy in the upstream file.

$$g(x) = (1 - e^{-(1-x)^2/\delta_x^2})(1 - e^{-x^2/\delta_x^2})$$
$$\psi_{\text{lid}} = (y-1)y^2 g(x) e^{-(1-y)^2/\delta_y^2} \tag{7}$$
$$\delta_x = \sqrt{10^{-3}}, \qquad \delta_y = \sqrt{10^{-2}}$$

At y=1, the lifting itself is zero, its x derivative is zero, and its y derivative is g(x). At y=0, the factor y squared makes both velocity components vanish. At x=0 and x=1, g and its first derivative vanish. Therefore the lifting prescribes u=g(x), v=0 on the lid, and zero velocity on the remaining walls, including the corners. These identities should be tested before optimization using arbitrary raw network outputs, not only after a trained model looks good.

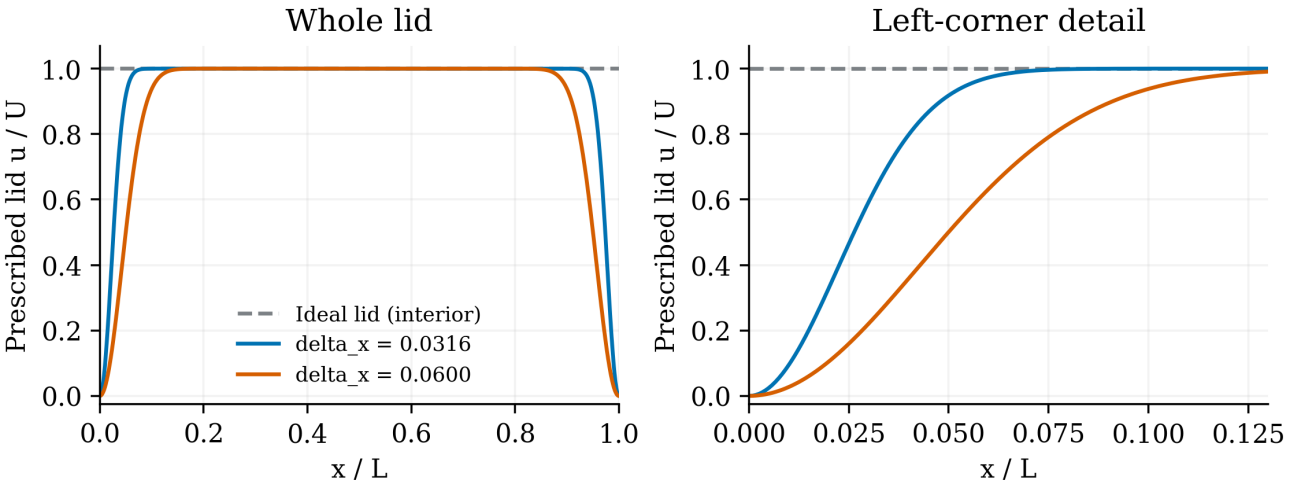


Figure 3. The analytic regularized lid profile. Blue uses the inspected upstream width; orange is an illustrative wider regularization, not another computed flow. The corner detail exposes a boundary-condition difference that a centerline plot alone could miss.

This is a regularized cavity, not the discontinuous unit-speed lid at its endpoints. Changing delta_x changes the boundary-value problem. Changing delta_y changes the interior extension of the boundary data, while leaving the prescribed lid velocity unchanged; it can nevertheless change optimization difficulty. A fair CFD reference must use the same g(x) and Re. The retained classical Re=100 field in Figure 1 is a physical illustration, not an error reference for an upstream Re=5000 regularized run.

One useful preflight is to evaluate the lifting on a dense boundary sample and compare its derivatives to the required wall velocities. A second is to vary the arbitrary correction and confirm that the wall values do not change. A third is to check the sign of the recirculation implied by u=psi_y and v=-psi_x. These inexpensive checks are more informative than immediately launching thousands of optimization steps.

7. What the inspected code actually does

The repository was inspected at commit fcb1566eaa3253d4a4108fbac9d49a38fd10ad6b. The following settings describe that snapshot, not all historical versions of DeepPlasma. The square-cavity file is a fixed-Re coordinate PINN, not the full geometry-parametric model discussed in the related research paper [3].

Table 2. Defaults read from the pinned upstream square-cavity script [1].

Ingredient	Inspected value	Scientific implication
Inputs / raw outputs	(x,y) / (q,p)	Fixed-Re field; velocity derived from a lifted streamfunction.
Architecture	3 hidden layers, 50 tanh units each	Two linear output channels; Xavier initialization.
Re / dtype / seed	5000 / float64 / 12345	Not an executed Re=20000 evidence record.
PDE / test points	262143 each	Independent random samples, same corner exclusion.
SOAP_STEPS	0	Optional stage disabled by default.
SSBroyden2	10000 outer iterations; strong-Wolfe search	Closure evaluations can exceed outer iterations.
Corner treatment	Exclusion radius 0.02; top-mask radius 0.01	Logged residual is not a full-domain raw residual.

The file contains alternative Sobol and Hammersley sampling functions inside a triple-quoted inactive block. The active functions use uniform random points and reject points within radius 0.02 of any corner. Merely seeing a function name in the file is not evidence that the training path uses it. Likewise, SOAP support exists, but SOAP_STEPS=0 disables that stage in the inspected defaults; describing the default run as SOAP followed by SSBroyden2 would be inaccurate.

The momentum residuals are additionally multiplied by smooth masks that taper near the two upper corners. If m is the product of the masks, minimizing the squared returned residual weights the raw momentum residual squared by m squared. Corner exclusion and residual weighting change where the optimization pays attention. The reported test loss uses the same residual function and corner-excluding sampler, so it is a held-out masked residual diagnostic, not an independent full-domain velocity error.

$$m = (1 - e^{-d_{\mathrm{TL}}^2/a^2})(1 - e^{-d_{\mathrm{TR}}^2/a^2}), \quad a = 0.01$$
$$\tilde{r}_u = m r_u, \quad \tilde{r}_v = m r_v, \quad \mathrm{L} = \langle m^2 (r_u^2 + r_v^2) \rangle \tag{8}$$

Report masked and unmasked residuals separately, with their domains. Use a second test set with deliberate wall and corner-band coverage; state any excluded region rather than hiding it in a single global number. Exclusion does not make a corner error disappear physically, and a small residual away from the corners does not establish wall-stress accuracy.

McDevitt's permission message reports that the case should be accurate up to Re=20000 and recommends GPUs with strong float64 performance, mentioning A100 and B200. These are author-supplied operating recommendations, not new FlowMLLab measurements. The default Re=5000 snapshot and the earlier parametric study [3] must not be presented as a locally reproduced Re=20000 result.

8. Optimization, precision, and a sensible experiment ladder

The objective is nonlinear and may be poorly conditioned. A first-order optimizer follows gradient information, while a quasi-Newton method constructs an approximation to curvature from changes between iterates. A line search can evaluate a closure several times within one outer iteration. Thus, comparing methods by iteration count alone is misleading: record wall time, residual evaluations, hardware, precision, and final independent errors.

When using a line search, keep its collocation set fixed within the step so that candidate steps are evaluated against the same function. The inspected SSBroyden2 stage fixes its PDE sample for the stage. If adaptive resampling is introduced, document when it occurs and how optimizer state is handled. A training curve after such a change is not automatically comparable to the original objective.

Begin on CPU with calculus and boundary tests. Then use an exact smooth Navier-Stokes solution to test the complete derivative and residual pipeline. After those gates, train a modest regularized cavity at Re=100 with a matched CFD reference. Increase Re only after the lower-Re field, wall, and grid checks are satisfactory. Continuation from a lower-Re checkpoint is a legitimate option, but record it: it is not the same experiment as training from a random initialization.

Float64 support and fast float64 arithmetic are different questions. Retain the upstream double-precision requirement initially and benchmark a small representative residual/backward pass on the allocated GPU. Record its model and peak memory before sizing the full batch. A CPU fallback being selectable does not make the default 262143-point third-derivative workload an appropriate laptop exercise. The optional SOAP path explicitly uses CUDA graphs and requires a compatible CUDA execution environment.

No cluster job is required to read or regenerate this lecture. The qualified PINN evidence discussed below was produced separately from a pinned upstream commit on Unity's gpu-preempt partition, with an environment record, atomic checkpoint, fixed evaluation protocol, and machine-readable audit. The lecture builder reads only the selected retained evidence; it never starts training on the reader's machine.

9. An exact test before a difficult cavity

The Kovasznay velocity-pressure field provides a smooth steady incompressible Navier-Stokes test without the cavity's discontinuous lid. On a chosen rectangular domain, prescribe its exact boundary values and use the following field. This is not a cavity solution and must not be labeled as one.

$$\lambda = \frac{Re}{2} - \sqrt{\frac{Re^2}{4} + 4\pi^2}$$

$$u = 1 - e^{\lambda x}\cos(2\pi y), \quad v = \frac{\lambda}{2\pi} e^{\lambda x}\sin(2\pi y) \tag{9}$$

$$p = \frac{1}{2}(1 - e^{2\lambda x})$$

Continuity cancels directly: u_x is minus lambda times the exponential-cosine factor, while v_y is plus the same quantity. For x momentum, convection reduces to minus lambda times the exponential-cosine factor plus lambda times exp(2 lambda x). The pressure derivative cancels the latter. The remaining coefficient vanishes because lambda squared minus Re times lambda minus 4 pi squared equals zero. The same identity cancels the y-momentum residual.

This gives an exact residual oracle before training. Evaluate the analytic derivatives in double precision and check the cancellation, then evaluate automatic derivatives of the same formulas. Only after those agree should the analytic field be replaced by a network. If the implementation cannot differentiate a known smooth solution correctly, increasing network width or training time cannot repair the underlying residual error.

When fitting a PINN to this benchmark, distinguish representation/optimization error from the roundoff-scale residual of the analytic formula. Compare on points not used for optimization. Boundary values are training information; the exact interior is an evaluation reference. Do not select the best checkpoint using the final evaluation set and then call its score blind.

10. What would count as an accurate cavity result?

Start by matching geometry, Re, lid profile, units, pressure gauge, and the assumption of steadiness. Then establish the reference solver's own convergence on at least three suitable resolutions. A neural field cannot be validated to an accuracy finer than an unresolved reference simply by reporting more decimal places. For classical cavity centerlines, Ghia, Ghia, and Shin [4] provide a widely used comparison, but their boundary convention must be distinguished from a regularized lid.

Use a common physical quadrature to compare the PINN and reference. Interpolate consistently and record the interpolation method. For pressure, subtract a weighted mean from each field over the same comparison domain. For velocity, a combined vector error avoids an unstable pointwise percentage near a zero crossing.

$$E_{\mathbf{u}} = \left[\frac{\sum_j w_j \|\mathbf{u}_{\theta,j} - \mathbf{u}_{\mathrm{ref},j}\|^2}{\sum_j w_j \|\mathbf{u}_{\mathrm{ref},j}\|^2} \right]^{1/2}$$

$$p_j^{\circ} = p_j - \frac{\sum_k w_k p_k}{\sum_k w_k}, \qquad w_j > 0 \tag{10}$$

Here w_j are positive quadrature weights and e_j is the vector velocity difference. Also report separate component errors, an absolute maximum error normalized by U, and centerlines u(0.5,y) and v(x,0.5). A relative pressure error needs a nonzero gauge-aligned reference norm; otherwise use an absolute pressure-scale error. Residual RMS, maximum wall mismatch, and net boundary flux diagnose different failure modes and should not be collapsed into one score.

Table 3. Complementary evidence; none replaces the others.

Check	What it diagnoses	What it cannot establish alone
Velocity field / centerlines	Agreement with a matched resolved reference.	Accuracy beyond the reference or at a different Re.
Wall velocity mismatch	Enforcement of the stated lid and no-slip condition.	Correct interior momentum balance.
Unmasked momentum residual	Differential balance on a declared independent support.	Uniqueness, stability or accurate wall conditions.
Divergence / boundary flux	Local/global kinematic consistency.	Correct vortex strength or pressure gradients.
Pressure after gauge alignment	Pressure variation relative to the reference.	A physically meaningful absolute pressure offset.
Seed / grid / sampling study	Sensitivity to initialization and numerical resolution.	Universality outside the tested problem family.

Inspect contours of reference, prediction, and signed or absolute error with equal geometric aspect ratios. Use identical ranges for reference and prediction; give the error map its own labeled range. Show wall-layer details separately instead of squeezing several unreadable panels into the main plot. A vortex-center estimate should be computed from the field, with a stated search method, not inferred from where a streamline picture appears darkest.

There is a useful logical warning: the identically zero velocity with constant pressure has zero steady momentum residual and zero divergence, but violates the moving-lid condition. Conversely, the hard lifting has correct wall velocities and continuity before training but need not satisfy momentum. These counterexamples explain why no single residual, physical invariant, or attractive contour is sufficient.

For research use, repeat independent initializations and retain failed runs as well as successful ones. A tighter future target might require combined velocity error below 1 percent at Re=100, with reference uncertainty comfortably smaller than that target and separately specified wall and residual limits. That is a proposed research target, not the gate used for the qualification below and not a universal PINN threshold. Revise a frozen criterion only with documented physical and numerical justification, not simply because a run fails.

10.1. Retained Unity qualification at Re=100

Before examining the trained field, the course protocol fixed three near-matched CFD gates: 10 percent for the vertical u centerline, 15 percent for the horizontal v centerline, and 15 percent for an interior velocity-vector comparison at 8,192 independently sampled points. The first 300-step run missed all three gates at 11.27, 19.24, and 15.51 percent. That failure was retained. No seed, collocation point, architecture, or threshold was selected after seeing it.

The run then continued the same float64 model, fixed set of 16,384 collocation points, dense SSBroyden2 inverse-Hessian state, and CPU/GPU random-number states to 1,000 steps. The continuation passed all three gates. It ran on an NVIDIA A100-SXM4-80GB in Unity's gpu-preempt partition. This supports a qualified Re=100 near-matched-reference claim; it does not establish high-Re accuracy or equivalence between the regularized and discontinuous lid problems.

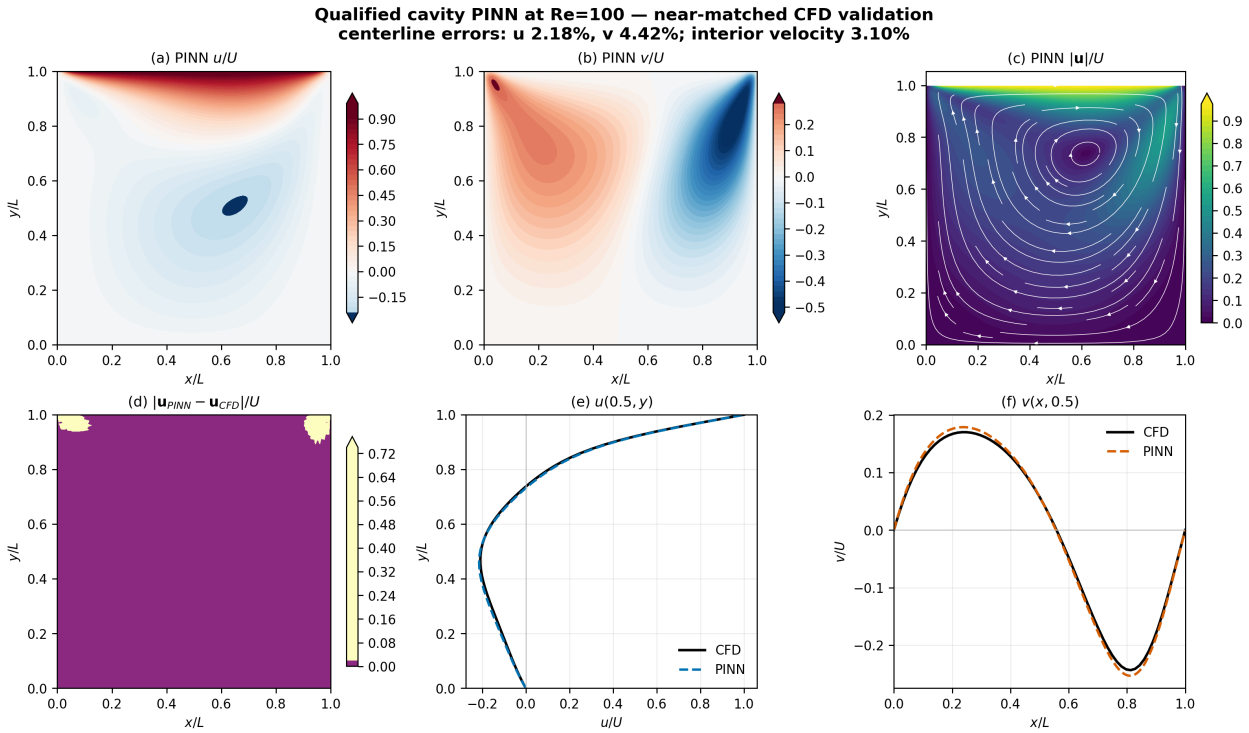


Figure 4. Qualified Re=100 PINN evidence regenerated from the accepted Unity checkpoint: velocity components, speed and streamlines, pointwise velocity error, and both CFD centerline comparisons. The large corner differences in panel (d) expose the regularized-versus-classical lid mismatch rather than being hidden by the domain-average metric.

Table 4. Predeclared Re=100 qualification against the near-matched CFD field.

Metric	Retained result	Frozen gate / decision
Vertical centerline u(0.5,y)	2.18% relative L2	10% / pass
Horizontal centerline v(x,0.5)	4.42% relative L2	15% / pass
Interior velocity vector	3.10% relative L2; 8,192 audit points	15% / pass
Hard velocity conditions	zero max error on 1,001 points per wall	reported separately
Unmasked momentum residual	RMS 0.0378 and 0.0246	reported separately

The independent raw residual audit used points distinct from the optimization set. Momentum residual RMS values were 0.0378 and 0.0246; divergence RMS was 2.34 times 10 to the minus 15. The hard lifting produced zero measured velocity mismatch on 1,001 points per wall. Field error, raw residual, and wall satisfaction remain separate evidence. The audit, optimizer history, exact software environment, failed first stage, and restart protocol are retained under results/week04_2_pinn_cavity.

11. From this forward solve to research questions

Once the fixed-Re problem is qualified, adding Re as an input creates a parameterized model only if the training objective also covers a specified range of Re. A network taking only x and y does not generalize across Re merely because the script contains a variable called Re. Report complete held-out Reynolds numbers, not random points from the same solved cases, when claiming parametric generalization.

McDevitt, Fowler, and Roy [3] studied a broader parameter space involving cavity geometry as well as Re. Their study distinguishes data-free training limitations from data-assisted modeling. It motivates asking where a few trusted observations improve a physically constrained representation, but it does not establish that the present square-cavity script reproduces every result of that study.

The local Vinuesa teaching material points to the RANS work of Eivazi and coauthors [5]. It is a useful later connection: sparse information and differential constraints can be combined to reconstruct flow quantities. Do not conclude that writing RANS residuals automatically resolves an underdetermined closure problem. State the observations and constraints that supply information, and analyze identifiability before interpreting inferred stresses as unique physical truth.

Three focused extensions are worth testing after the baseline. First, compare soft and hard wall constraints at matched evaluation accuracy and total computational cost. Second, vary the lid regularization width while regenerating the matched reference. Third, compare uniform and wall-enriched sampling with an unchanged independent test distribution. Each experiment should change one declared ingredient and explain whether it changes the physical problem, the approximation space, or only the optimizer.

12. Guided assignment and worked reasoning checks

Assignment A: derive the dimensionless residuals from dimensional momentum and write a one-page specification. Include the lid profile, corner treatment, pressure gauge, Re, sampling domain, and acceptance metrics. A correct answer explains why pressure uses rho U squared and why the coefficient of the viscous term is 1/Re; listing the final equations without the scales is incomplete.

Assignment B: replace B squared in the hard lifting by B and evaluate a wall derivative. The correction to tangential velocity generally contains a nonzero derivative of B multiplied by q_theta. Therefore the replacement does not preserve no slip for arbitrary network output. Confirm the counterexample using q_theta=1; do not rely on a trained network that might accidentally make the error small.

Assignment C: implement the exact Kovasznay field and compare analytic and automatic first and second derivatives. Deliberately exchange x and y derivatives in one momentum term. The exact-solution test must fail. Record the defect, the failed check, and the corrected implementation. This connects the scientific-software audit of Week 1.1 to PINN residual construction.

Assignment D: plan a matched Re=100 CFD/PINN comparison before running it. Freeze the reference generation protocol and reserve an evaluation grid. Submit a table with velocity error, gauge-aligned pressure error where available, wall mismatch, unmasked residual, runtime, and seed. Explain which measurements would remain necessary if the training loss were reduced by another factor of one hundred.

Assignment E: audit the pinned DeepPlasma file. Identify the two raw outputs, the active sampler, the disabled SOAP stage, the masked residual, and the fixed training sample in SSBroyden2. Explain why the variables named TrainLoss1 and TrainLoss2 refer to the two momentum equations rather than to separate physics and boundary losses. Reading executed paths is part of scientific reproducibility.

The final short report should state: what physical problem was specified, what the network was allowed to represent, what the optimizer minimized, and what the independent evidence supports. If a run fails, a precise account of that failure is more valuable than a favorable plot from an undocumented seed. The objective is to understand and qualify a numerical method, not to declare that neural networks replace CFD.

References and provenance

[1] C. J. McDevitt, DeepPlasma, LDC/LDC_module_square.py. Code snapshot inspected: fcb1566eaa3253d4a4108fbac9d49a38fd10ad6b. https://github.com/cmcdevitt2/DeepPlasma/tree/fcb1566eaa3253d4a4108fbac9d49a38fd10ad6b/LDC . Case use permitted by McDevitt to Ehsan Roohi. Linking and discussing the implementation does not change its upstream license; the external source is not redistributed in this lecture package.

[2] M. Raissi, P. Perdikaris, and G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics 378 (2019), 686-707. https://doi.org/10.1016/j.jcp.2018.10.045 . Foundational PINN formulation.

[3] C. J. McDevitt, E. Fowler, and S. Roy, Physics-Constrained Deep Learning of Incompressible Cavity Flows (2022). https://arxiv.org/abs/2211.06375 . Related cavity research, distinct from the inspected fixed-Re script.

[4] U. Ghia, K. N. Ghia, and C. T. Shin, High-Re solutions for incompressible flow using the Navier-Stokes equations and a multigrid method. Journal of Computational Physics 48 (1982), 387-411. https://doi.org/10.1016/0021-9991(82)90058-4 . Classical cavity benchmark, not a regularized-lid reference by default.

[5] H. Eivazi, M. Tahani, P. Schlatter, and R. Vinuesa, Physics-informed neural networks for solving Reynolds-averaged Navier-Stokes equations. Physics of Fluids 34 (2022), 075117. https://doi.org/10.1063/5.0095270 . Related public manuscript: https://arxiv.org/abs/2107.10711 . Further reading on data-assisted flow reconstruction.

Local pedagogical resources consulted: Ricardo Vinuesa's 1_Time_PINNs.pdf; the Mastering CFD PINNs series, particularly the Stokes, Navier-Stokes, and exact-validation progression; and the PINNs from Scratch in PyTorch notebook in the instructor's teaching collection. These informed topic selection, not copied text, code, or figures. Authorship and redistribution rights for the locally supplied worksheet collection were not established, so those files are not included. The local Burgers dissertation was screened but is not used as an accuracy authority for cavity flow.

Figure 1 uses the existing accepted Re=100 field in FlowMLLab data/cavity_data.npz; it is conventional CFD, not a newly trained PINN. Figures 2 and 3 are original explanatory graphics generated from the stated construction. The companion build script records the CFD archive hash and evaluates analytic preflight identities. Those checks validate the illustrated formulas, not the convergence of an upstream training run. This working-course lecture does not create a new software release or DOI.

Executable derivative preflight

The Week 13 core notebook now performs the required analytic Kovasznay test before training: exact first derivatives and steady momentum balance at Re=40, metric checks at D=1 and D=2.2, a deliberately omitted metric factor that fails at D=2.2, and all four cavity walls with random untrained weights. Read the analytic formulas, predict the failure, then run each gate. The sampler recomputes the corner mask if Adam points change; L-BFGS keeps a fixed objective. A passed preflight verifies implementation, not the learned cavity solution.